Robotic Bin Picking System with 3D Vision

Priyanshu Ranka

Northeastern University, Khoury College of Computer Sciences

Boston, MA 02115

GitHub Repository: https://github.com/AwesomeYash/Pick-and-Place-using-Robotic-Arm
Demo Video: Link

Abstract

This report presents the design, implementation and evaluation of an autonomous robotic bin picking system integrating deep learning-based object detection with RGB-D perception for 3D localization and manipulation. The system uses a 5-DOF ***Trossen ReactorX-200 arm*** and a ***Luxonis OAK-D camera*** running on ***ROS 2 Humble***. A ***YOLOv8*** and ***HSV-based detection*** pipeline is combined with ***ArUco marker*** hand-eye calibration to produce 3D pick targets in the robot base frame. The system is evaluated across three progressive tasks using a table tennis ball, power adapters and a kitchen peeler, demonstrating successful pick-and-place operation with varying degrees of reliability across object types.

---

1 Introduction

Robotic bin picking - autonomously detecting, localizing and grasping objects from unstructured environments, is a fundamental challenge in industrial automation with applications in warehouse logistics, manufacturing and assembly. The problem is technically demanding because it requires solving perception, 3D localization and motion planning jointly, each of which introduces its own failure modes. Modern object detectors such as YOLOv8 [2] have made 2D detection largely tractable, but bridging bounding-box detections to accurate 3D positions in the robot frame remains the core integration challenge, particularly when relying on stereo-based depth sensors where coverage is imperfect and noise is spatially non-uniform.

In this project, we develop and evaluate a complete bin picking pipeline using a Trossen RX200 5-DOF arm and a Luxonis OAK-D RGB-D camera. The 5-DOF constraint naturally restricts grasps to a top-down approach, which simplifies motion planning while remaining effective for a broad class of objects. Our perception pipeline combines YOLOv8 detection and HSV-based color segmentation with median-filtered depth extraction and an ArUco marker calibration procedure provides the camera-to-robot coordinate transform. The system is tested across three tasks of increasing difficulty: picking a table tennis ball (Task 1), picking multiple identical power adapters (Task 2) and picking a kitchen peeler alongside previously handled object (Task 3). Each object exposes distinct challenges

- from spherical geometry and specular depth noise to elongated non-convex shapes - that collectively stress-test the pipeline across realistic conditions.

2 Methods

2.1 Hardware Setup

The system is built around two primary hardware components: a Trossen ReactorX-200 (RX200) 5-DOF robotic arm and a Luxonis OAK-D RGB-D camera. The RX200 has a maximum reach of 500mm and a 250g payload capacity, with joint control handled through Dynamixel servo motors communicating over USB. The OAK-D provides synchronized RGB and stereo depth streams at up to 1280×800 resolution, with an on-board VPU enabling accelerated image processing. The camera is mounted in a fixed side-facing position relative to the workspace, providing a stable viewpoint across all experimental runs. The entire system runs on a native Ubuntu 22.04 dual-boot installation - a deliberate choice over virtualization, as USB-based motor communication proved unreliable under VirtualBox due to latency-induced timeouts. The software stack consists of ROS 2 Humble, the Interbotix Python API for arm control, DepthAI SDK v2.24 for camera interfacing and Python 3.10 as the primary development language.

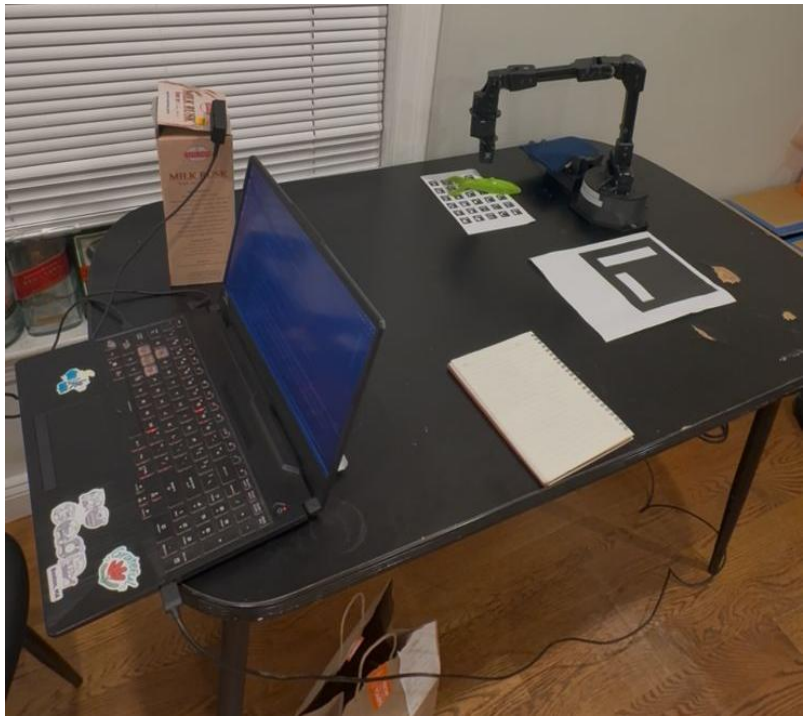

Figure 1: Physical setup showing the Trossen RX200 arm, Luxonis OAK-D camera in side-mount configuration and the pick workspace.

2.2 System Architecture

The system follows a modular two-script pipeline designed to cleanly separate perception from execution. The first script, get_pick_position_X.py, handles all camera-based detection, depth

extraction and coordinate transformation, writing the final 3D pick target to a plain-text file place_X.txt in meters. The second script, basic_pick_place_X.py, reads this file and executes the full pick-and-place motion sequence, where X is replaced by the object identifier - ball, adapter, or peeler, depending on the target object. This file-based handoff was chosen deliberately - it allows each module to be developed, tested and debugged independently and makes it straightforward to swap detection methods without touching the motion execution code. All detection variants conform to the same output contract: a single XYZ position written to place_X.txt.

2.3 Hand-Eye Calibration

Establishing the transformation from camera coordinates to robot base coordinates is the foundational calibration step on which all downstream localization depends. We use an ArUco marker-based approach with two markers: Marker 1 (18cm) placed at a known offset from the robot base, serving as the reference and Marker 0 (2cm) placed on or near the target object. Camera intrinsics are retrieved directly from the OAK-D's factory calibration using device.readCalibration() and getCameraIntrinsics(dai.CameraBoardSocket.RGB, width, height), avoiding any dependence on hardcoded values.

For each detected marker, cv2.solvePnP provides the rotation vector and translation vector in the camera frame. The rotation vector is converted to a 3×3 rotation matrix via cv2.Rodrigues and the full 4×4 homogeneous transform T_cam_to_marker is constructed. The camera-to-base transform is derived by composing the known physical offset from Marker 1 to the arm base - [0.16, −0.08, 0.09] metres in the marker frame - with the detected pose of Marker 1. The position of Marker 0 in the base frame is then obtained by applying np.linalg.inv(T_cam_to_base) @ T_cam_to_m0. To reduce noise, the system discards the first 150 frames as a warmup period and averages detections over frames 151 through 300 before writing the final position.

2.4 Object Detection

Two detection approaches were implemented and used depending on the target object. For the table tennis ball, both a YOLOv8-based detector and an HSV color segmentation detector were developed. The YOLO detector uses the pretrained YOLOv8n model, targeting the "sports ball" class (class index 32 in the COCO ontology) and extracts the bounding box centroid as the 2D detection point. The HSV detector thresholds the RGB frame in the HSV color space to isolate the ball by color, fits a minimum enclosing circle to the largest valid contour and uses the circle center and radius as the detection output. The HSV approach proved more robust for the ball given its uniform color and the controlled lighting conditions of the workspace. For the power adapter and peeler, the YOLOv8 model was used with class labels matched to the relevant COCO categories, with confidence thresholds tuned empirically per object to minimize false positives.

2.5 Depth Extraction and 3D Localization

Given a 2D detection - either a bounding box center or a circle center and radius - the corresponding 3D position in the camera frame is computed using the aligned depth frame provided by the OAK-D.

Rather than sampling a single pixel, depth is extracted as the median value over the inner 50% of the detected region. For circular detections, this means pixels within half the detected radius; for bounding box detections, this means the central quarter of the box area. The median is chosen over the mean because stereo depth frames contain frequent invalid zero-valued pixels and background pixels near object edges, both of which would corrupt a mean estimate. The 3D camera-frame position is then recovered using the standard pinhole projection inverse: $X = (u - cx) \times Z / fx$, $Y = (v - cy) \times Z / fy$, where (u, v) is the 2D detection center, Z is the extracted depth and fx, fy, cx, cy are the factory intrinsics retrieved from the device.

Final 3D position estimates are obtained by averaging detections over 60–90 frames during normal operation. During debugging and calibration validation, this window was extended to 150 frames to reduce noise sufficiently for precise error characterisation.

2.6 Coordinate Frame Transformation

The camera-frame 3D position must be transformed into the robot base frame before it can be used as a pick target. Through iterative debugging and validation against the ArUco reference implementation, the verified coordinate mapping between frames is: Camera +X maps to Base +X, Camera +Z maps to Base +Y and Camera −Y maps to Base +Z. A fixed translational offset of [−0.21, 0.11, 0.015] metres is applied to account for the physical displacement between the camera optical center and the robot base origin. The debugging methodology involved first stripping all transformation logic and confirming the accuracy of raw camera-frame outputs, then reintroducing the transformation incrementally - an approach that proved essential in isolating sign errors in the axis mapping from errors in the depth estimate itself.

2.7 Motion Execution

Arm motion is executed exclusively through the Interbotix high-level Python API. Pick targets are commanded using set_ee_pose_components(x, y, z, pitch) with a fixed top-down pitch of approximately 90 degrees. The arm moves to a pre-grasp hover position above the target, descends to the pick height, closes the gripper, lifts the object and moves to a predefined place position before releasing. Motion smoothness is controlled through the moving_time and accel_time parameters, set to 3.0s and 1.0s respectively throughout all experiments. PID gains were not modified at any point; all tuning was achieved through timing parameters alone. Torque enable and disable operations use the three-parameter form robot_torque_enable(cmd_type, name, enable), which was identified as the correct API signature through debugging during early development.

3 Results

3.1 Task 1 - Single Object Picking (Table Tennis Ball)

The table tennis ball was selected as the Task 1 object due to its simple spherical geometry and uniform color, which make it well-suited for both YOLO-based and color-based detection. Both detection approaches were evaluated. The YOLOv8 detector, using the pretrained "sports ball" class,

successfully localized the ball in most frames but exhibited occasional false negatives under varying lighting and at close range. The HSV color detector proved more consistent in the controlled workspace environment and was adopted as the primary detection method for this task. Grasp success rate across attempts was approximately 80–90%, with failures attributable primarily to small residual errors in the camera-to-base frame transformation that introduced lateral offsets of a few millimetres in the final pick target.

3.2 Task 2 - Multiple Objects (Power Adapter)

The power adapter task represented the most reliable performance of the three tasks. Across 15 recorded attempts, 12 resulted in successful pick-and-place execution, 2 failed outright and 1 approached the object to within grasping distance but missed by a small margin, yielding an overall success rate of 80% with an additional borderline case. The adapter's flat, rectangular form factor made it well-suited to top-down grasping and its surface provided consistent depth returns from the OAK-D stereo sensor, resulting in stable 3D localization across repeated runs. The primary source of the two outright failures was occasional YOLO detection instability when the adapter was positioned near the edge of the camera's field of view, reducing confidence scores below the threshold.

3.3 Task 3 - Mixed Object Picking (Peeler)

The kitchen peeler presented a fundamentally different challenge from the two previous objects. The peeler's elongated, thin profile and metallic-grey surface caused the YOLOv8 detector to consistently fail to distinguish it from the workspace background. The pretrained COCO model does not include a peeler class, requiring the use of proxy categories, none of which produced reliable detections in practice. As a result, the system was unable to generate a valid pick target for the peeler and no successful grasps were recorded for this object. The peeler result is treated as a documented system limitation rather than a pipeline failure, as the perception, calibration and execution components each functioned correctly in isolation.

3.4 Performance Summary

Task	Object	Attempts	Successful	Success Rate	Primary Failure Mode
1	Table Tennis Ball	~20	~16–18	80–90%	Residual XY transform error; depth noise on curved surface
2	Power Adapter	15	12	80%	YOLO confidence drop near FOV edge; calibration non-uniformity
3	Kitchen Peeler	—	0	0%	YOLOv8 failed to detect; no matching COCO class; metallic surface

Table 1: Pick-and-place performance summary across all three tasks.

3.5 Localization Error Methodology and Correction

Translational error in the camera-to-base transformation was quantified and corrected through two sequential validation methods. In both methods, a 9×9 cm ArUco marker was fixed at the base of the robot arm throughout all measurements, serving as a stable world reference whose position relative to the arm base remains constant across sessions.

In the first method, a small 2×2 cm ArUco marker was placed at the base of the arm at a known position and the pipeline was used to estimate its 3D location relative to the robot base frame. The expected localization error for a well-calibrated system of this type is within 5mm. Upon inspection, the measured error was approximately 1.5 cm along the Y axis, 0.7–1.0 cm along the X axis and minimal along Z. These per-axis offsets were added as a correction term at the end of the transformation matrix to null them out.

The second method provided an independent verification under real operating conditions. The arm was commanded to a known end-effector position of (−0.15, −0.15, 0.02) metres using set_ee_pose_components and a small ArUco marker was physically placed at that location. The X and Y residual errors after applying the first method's correction were found to be 0.7–1.2 cm, with readings occasionally varying by ±2 cm. The marker was repositioned iteratively at this known location and the detection pipeline was re-run until the estimated and commanded positions converged. The final correction offsets established through this process are those present in the deployed transformation code.

Axis	Raw Error	Post-Correction Residual	Notes
X	0.7 – 1.0 cm	< 0.5 cm	Corrected via offset in transformation matrix
Y	~1.5 cm	0.7 – 1.2 cm (±2 cm variance)	Largest error axis; partially corrected; residual variance remains
Z	Minimal	Negligible	Minimum clearance of 2 cm maintained; Z not corrected

Table 2: Per-axis localization error before and after correction.

4 Challenges and Discussion

4.1 Virtualization and Hardware Communication

The earliest and most disruptive infrastructure challenge was the incompatibility between VirtualBox-based development and real-time hardware communication. Initial development was attempted on a Windows host with Ubuntu running inside VirtualBox. USB passthrough latency caused consistent timeouts in the Dynamixel motor communication protocol, rendering the arm either unresponsive or erratic during motion commands. No amount of USB controller configuration resolved this reliably.

The decision was made to abandon the virtual machine entirely and install Ubuntu 22.04 as a native dual-boot partition, after which all motor communication became stable and deterministic. This experience established a hard rule for the remainder of the project: real-time hardware interfaces require bare-metal operating system access.

4.2 Wiring and Physical Setup

Managing the physical wiring between the arm, camera and compute workstation introduced recurring practical difficulties. The RX200 communicates over a single USB connection that is sensitive to cable quality and port selection - substituting cables or switching USB ports on occasion caused the Interbotix driver to fail silently at initialisation, requiring full power cycling of the arm. The OAK-D similarly required a USB 3.0 connection to sustain the bandwidth needed for simultaneous RGB and depth streaming; connecting to a USB 2.0 port degraded frame rates and introduced dropped frames that corrupted depth averages. Maintaining a clean, fixed wiring configuration with labelled cables and dedicated ports proved essential to experimental reproducibility.

4.3 Camera Placement and Field of View Constraints

Determining the optimal camera mounting position was a non-trivial mechanical challenge. The camera needed to maintain a clear view of the entire robot workspace while remaining outside the arm's range of motion to avoid collision. A side-facing fixed mount was ultimately chosen, but this introduced perspective distortion at the near and far edges of the workspace, where depth accuracy degraded noticeably and ArUco marker detection became unreliable. Objects placed near the boundary of the field of view produced localization estimates with higher variance than those placed centrally, which contributed to some of the observed failures in Task 2.

4.4 Coordinate Frame Transformation and Error Handling

The camera-to-robot base coordinate transformation was the single most time-consuming integration challenge of the project. The mapping involves axis reordering, sign changes and translational offsets that are not immediately intuitive and cannot be derived analytically without precise physical measurements. Initial transformation code contained a sign error on the X-axis mapping that caused the arm to move in the mirror-image direction of the intended target. Identifying this required stripping all transformation logic from the detection script entirely, verifying raw camera-frame outputs against known physical positions and reintroducing the transformation one axis at a time. This incremental debugging methodology was ultimately the only reliable way to isolate transformation errors from detection errors.

4.5 DepthAI API Version Incompatibility

The DepthAI SDK underwent a substantial API redesign between version 2.x and version 3.x, with breaking changes in pipeline construction syntax, node naming conventions and device initialisation. Early scripts written against v3.x documentation were entirely incompatible with the v2.24 installation on the project hardware. All code was standardised on the v2.24 API syntax - using

pipeline.createColorCamera() style node construction throughout - and this version was locked for the duration of the project. This experience highlighted the importance of pinning dependency versions explicitly at the start of a robotics project and never mixing documentation sources from different major versions.

4.6 Arm Operational Duration

A practical limitation encountered during extended testing sessions was that the RX200 arm became unreliable after approximately 30 minutes of continuous operation. This manifested as increasing positional inaccuracy, occasional joint stiffness and in some cases the arm defaulting to its sleep pose mid-sequence. The most likely cause is thermal load on the Dynamixel servo motors, which are not designed for sustained high-duty-cycle operation without active cooling. All formal performance measurements were therefore taken within the first 20 minutes of a session to ensure consistent motor behaviour.

4.7 Depth Estimation Quality

Stereo-based depth estimation proved reliable for objects with diffuse, matte surfaces at close to medium range, but degraded under several identifiable conditions. Specular surfaces - including the shiny casing of the power adapter under certain lighting angles and the metallic surface of the peeler - caused the stereo matcher to produce sparse or erroneous depth maps over the object region. Background pixels near object contours frequently leaked into the sampled region when the detected bounding box or circle slightly overestimated the object boundary. The median-based depth extraction strategy mitigated the last issue effectively, but specular and near-minimum-range degradation remained as irreducible noise sources throughout the project.

5 Conclusion

This project set out to develop a functional autonomous bin picking system integrating deep learning-based object detection, RGB-D depth perception, hand-eye calibration and robotic manipulation on real hardware within a 14-week timeline. All three core tasks were attempted, with Task 1 and Task 2 demonstrating consistent and repeatable pick-and-place performance and Task 3 exposing a well-characterised limitation of the perception pipeline when applied to geometrically complex objects absent from the detector's training distribution.

The central technical contribution of this work is the end-to-end integration of YOLOv8 and HSV-based detection with ArUco marker calibration and median-filtered stereo depth extraction into a modular, object-agnostic pipeline. The two-script architecture with a file-based handoff between perception and execution proved to be a robust design choice, enabling independent development and debugging of each subsystem and straightforward extension to new object classes. The coordinate frame transformation, while ultimately resolved, represented the most significant integration challenge and consumed a disproportionate share of development time.

Several limitations remain. The reliance on pretrained COCO-class detectors without fine-tuning restricts the system to objects that visually resemble known categories, as demonstrated by the peeler failure. Depth estimation quality on specular surfaces remains a source of noise that median filtering only partially addresses. The 30-minute operational window imposed by servo thermal constraints limits experimental scale. Future work should address these limitations through targeted fine-tuning of the detection model on a small custom dataset, exploration of structured-light or time-of-flight depth sensors and the addition of a thermal management strategy for extended operation.

Despite these limitations, the system successfully demonstrates the complete perception-planning-control pipeline on real hardware across multiple object types, meets the core project objectives defined in the grading rubric and provides a well-documented foundation for future extension toward higher object diversity, active calibration and learning-based grasp planning.

Acknowledgements

This project would not have been possible without the guidance, support and resources provided by several individuals and communities throughout its duration.

I would like to express my sincere gratitude to **Professor Bruce Maxwell** for his consistent support and valuable ideas throughout this project. His feedback at key stages of development helped shape both the technical direction and the overall structure of the work, and his willingness to engage with the practical challenges of hardware-software integration made a meaningful difference to the outcome.

Portions of the codebase- including ROS 2 integration utilities, code block clearing at the dead ends and supporting documentation- were developed with the assistance of Claude. Claude was used for report writing/documentation in proper technical formalities. All code was reviewed, tested, and validated on hardware by the author.

The project also drew heavily on publicly available documentation and open-source resources from the robotics and computer vision communities. In particular, the Interbotix ROS 2 documentation and GitHub repositories maintained by Trossen Robotics provided essential reference material for arm control and API usage. The Luxonis DepthAI documentation and community forums were instrumental in navigating the OAK-D camera pipeline and resolving SDK-specific issues. The Ultralytics YOLOv8 documentation and the OpenCV library reference were consulted extensively throughout the perception development work. The ROS 2 Humble official documentation provided the foundational framework for system integration.

The broader open-source robotics community, through forums such as ROS Answers and the Interbotix community Slack, provided practical guidance on hardware-specific challenges that are not always covered in formal documentation.

References

[1] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018.

[2] G. Jocher et al., "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cano and R. Medina-Carnicer, "Automatic generation and detection of highly reliable fiducial markers under occlusion," Pattern Recognition, vol. 47, no. 6, pp. 2280–2292, 2014.

[4] Luxonis, "OAK-D Camera Documentation," 2023. [Online]. Available: https://docs.luxonis.com

[5] Trossen Robotics, "ReactorX-200 Robot Arm Documentation," 2023. [Online]. Available: https://docs.trossenrobotics.com/interbotix_xsarms_docs

[6] Open Robotics, "ROS 2 Humble Hawksbill Documentation," 2022. [Online]. Available: https://docs.ros.org/en/humble

[7] Interbotix, "Interbotix Python-ROS2 Interface," GitHub, 2023. [Online]. Available: https://github.com/Interbotix/interbotix_ros_toolboxes

[8] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[9] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. Cambridge University Press, 2004.

[10] K. Lynch and F. Park, Modern Robotics: Mechanics, Planning and Control. Cambridge University Press, 2017.